Lecture 5: Non-Parametric Classification

Spring 2026

10 ECTS Credits

Department of Information and Communication Technology

Faculty of Engineering and Science

UiA

Recap

- Maximum Likelihood Estimation
  - What is a parameter?
  - Likelihood Function
- Parametric Classification
- Parametric Plug-in Rules
- Gaussian Discriminant Analysis
  - Linear Discriminant Analysis
  - Variants of LDA
  - Quadratic Discriminant Analysis
- Logistic Classification

Parametric Classification

- We estimate the parameters of a distribution (assumed or known) that represents the data.
- We use Maximum Likelihood Estimation to estimate the parameters.
- Data fits to common density functions such as normal, Poisson, geometrical, etc.
- We are happy ☺
- However, data distribution is sometimes too irregular and doesn't fit to any of the usual PDFs.
- We have to look at non-parametric classification.

Non-Parametric Classification

- No assumption about the shapes of the distribution are made.
- Distributions are approximated using a **smoothing** process.
- Smoothing:
  - modifies raw data,
  - helps in reducing noise and highlight important patterns,
  - achieved by averaging or weighting nearby datapoints.
- To obtain the approximations of:
  - Class-conditional densities $\left(p_n(\boldsymbol{x} \mid Y = 0) \text{ and } p_n(\boldsymbol{x} \mid Y = 1)\right)$, or
  - Posterior probability function $\eta_n(\boldsymbol{x}) = P_n(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$.
- Finally plugged in to the definition of the Bayes Classifier.

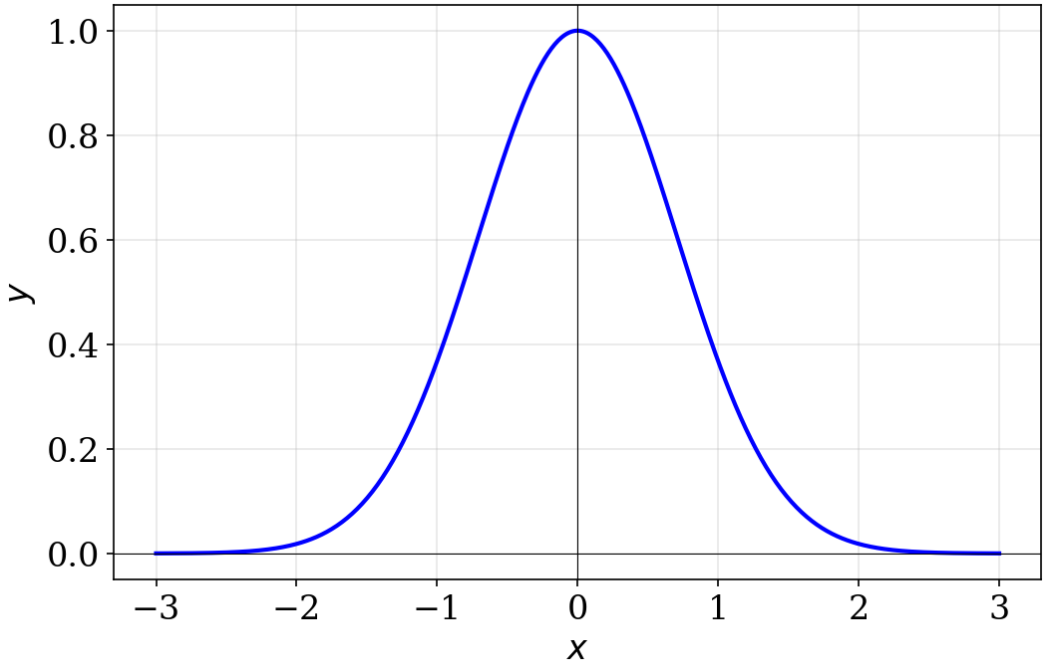
Kernel Density Estimation

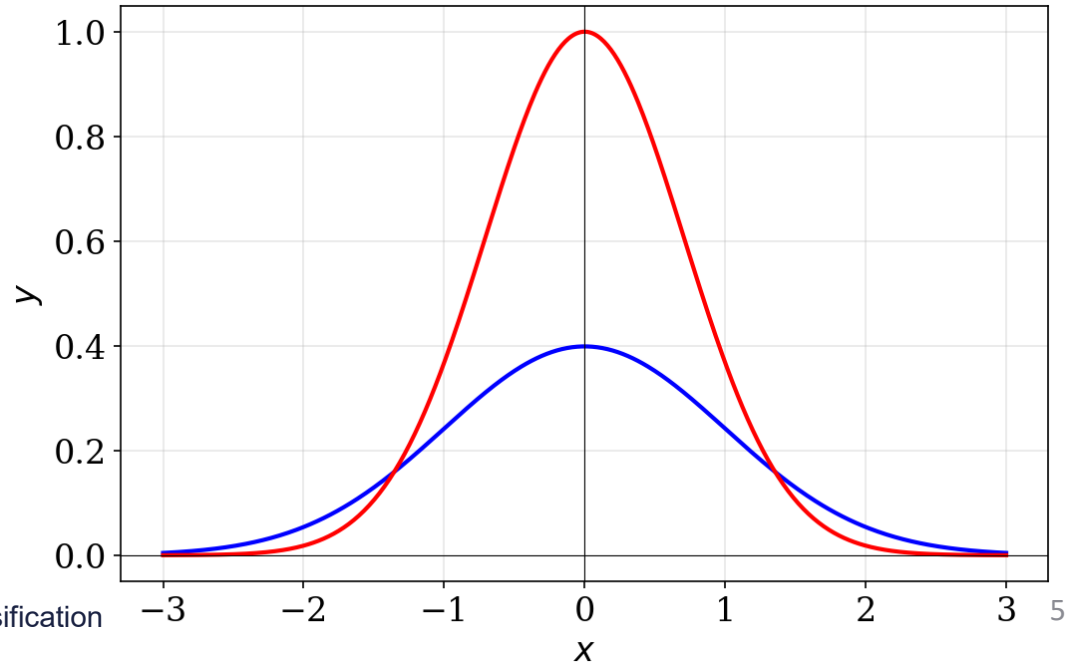
- KDE is a function made of a single type of building block known as kernels.
- The key property is that these kernels can shift, stretch, or shrink.
- Consider a data point $x = 0$ and create a PDF for this single point.

$$y = f(x) = \exp(-x^2)$$

- But PDF is supposed to have unit area under the curve: (Why?)

$$y = f(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$





Kernel Density Estimation

- Thus, our building block (kernel) is

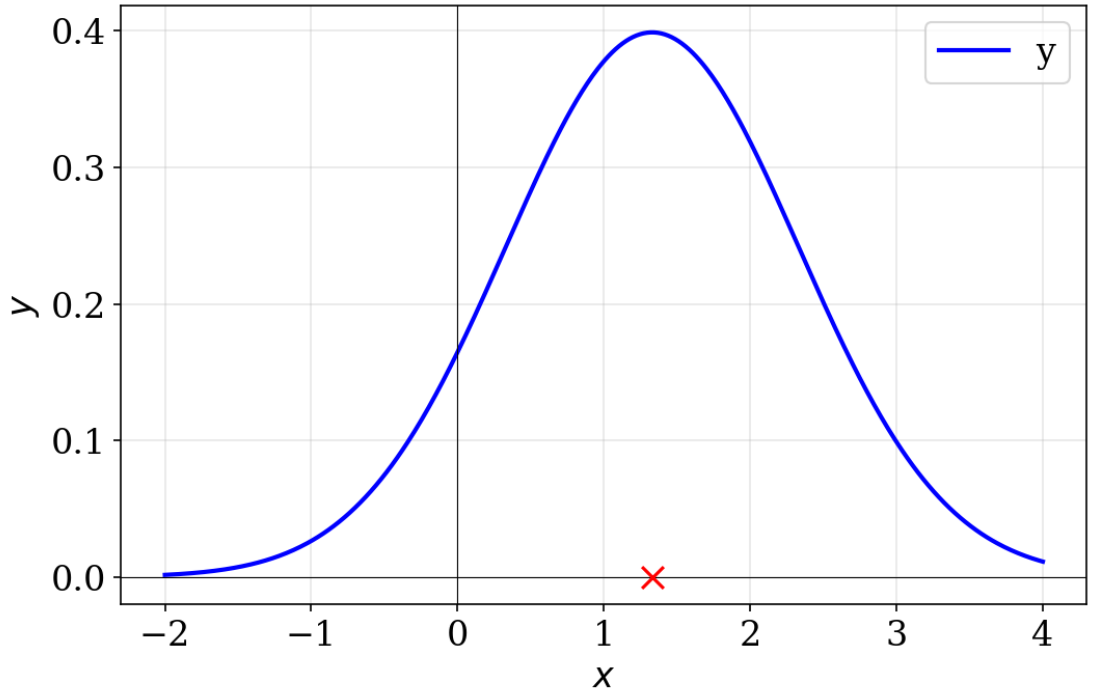
$$K(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

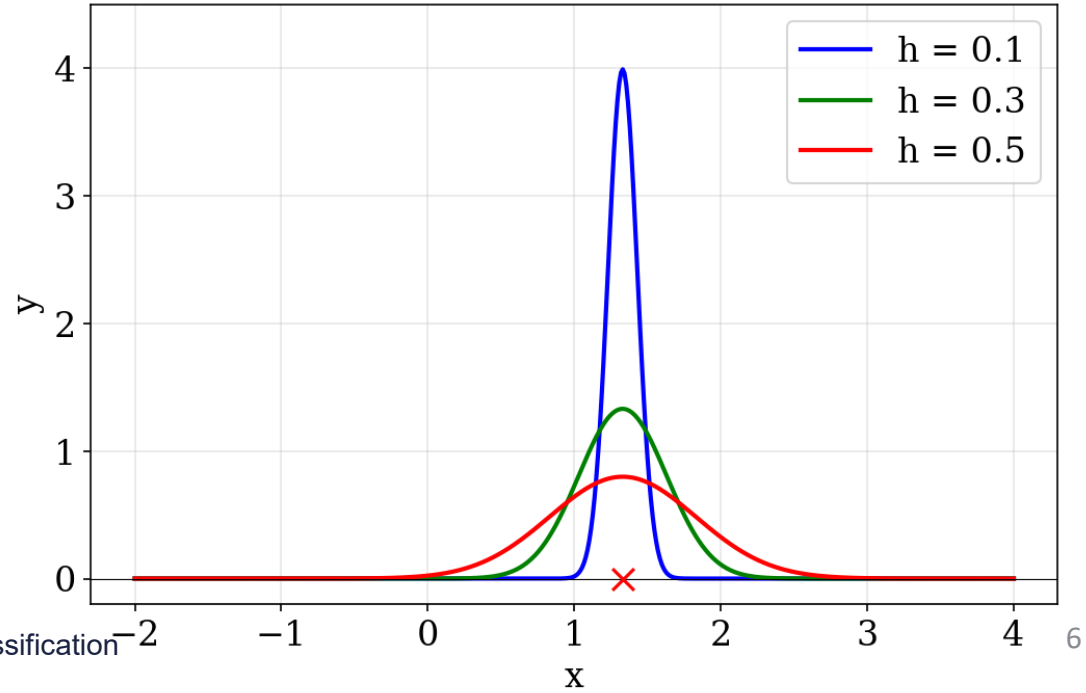
- Equivalent to Gaussian distribution with zero mean and unit variance.
- Shifting the kernel to a new data point.

$$K(x - x_i)$$

- Now to stretch or shrink:

$$\frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$





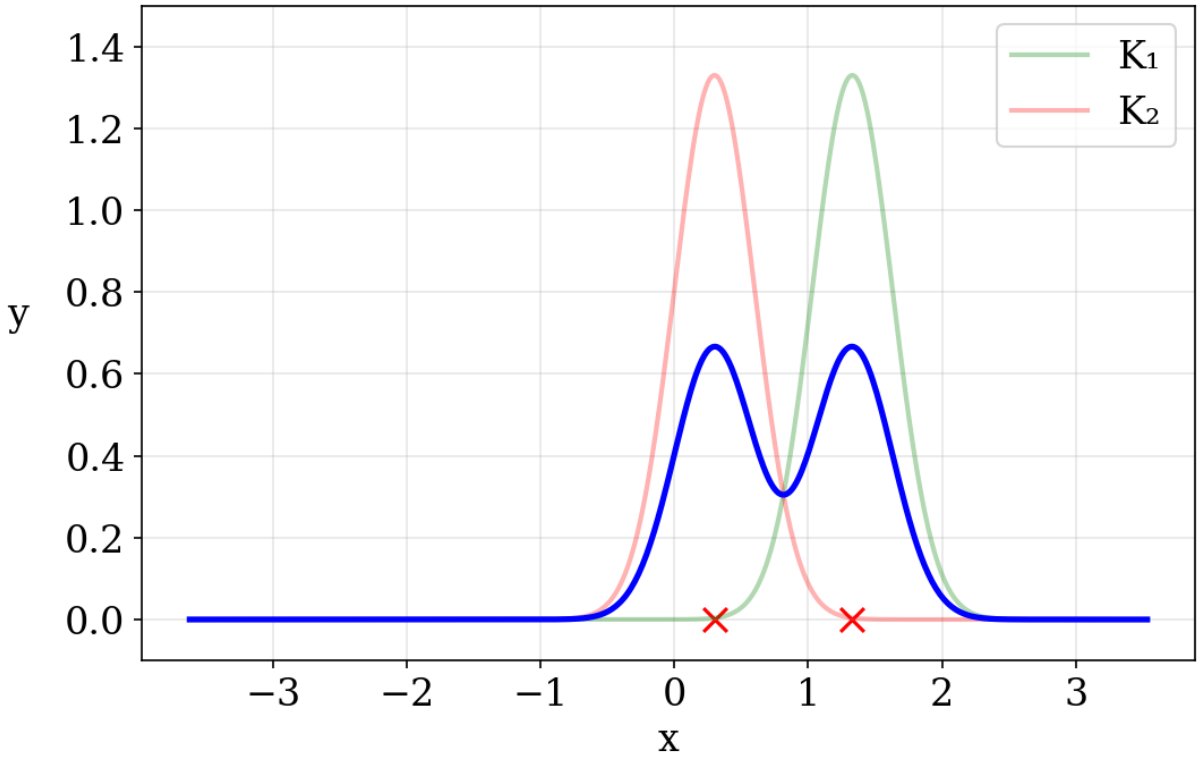
Kernel Density Estimation

- Given a dataset
  $x = [1.33, 0.3, 0.97, 1.1, 0.1, 1.4, 0.4]$
- We take $h = 0.3$
- For the first data point, we calculate:

$$\frac{1}{h} K\left(\frac{x - x_1}{h}\right)$$

- Similarly for second data point and add them:

$$\frac{1}{2h} \sum_{i=1}^{2} K\left(\frac{x - x_i}{h}\right)$$

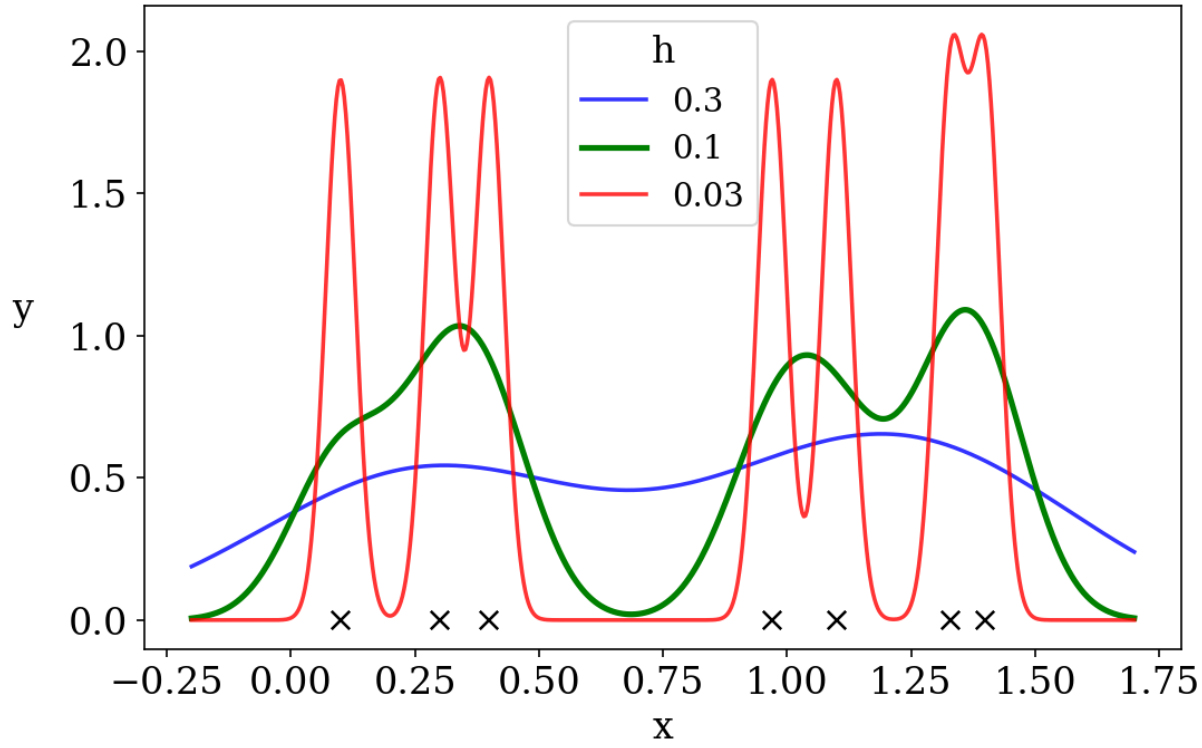


Kernel Density Estimation

- Generalizing for n data points

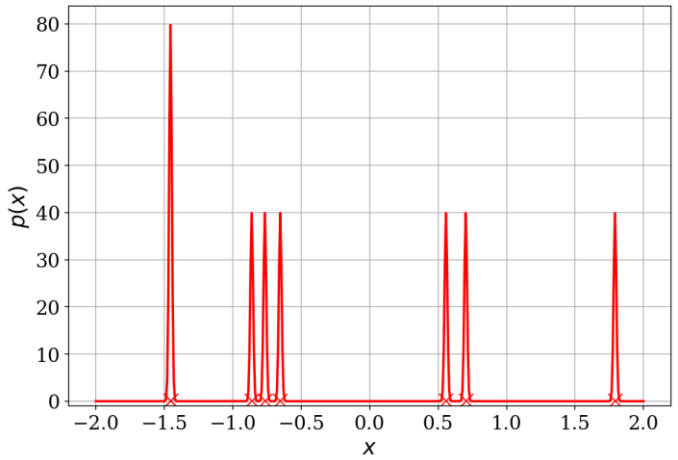
$$\frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

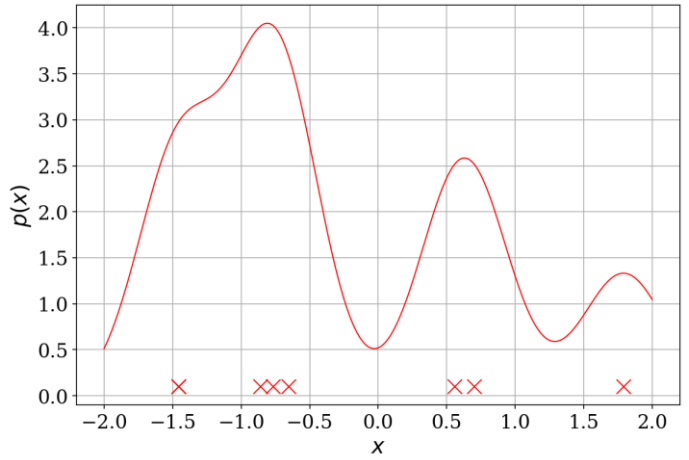
- KDE could be used for both estimating class-conditional densities $p_n(\boldsymbol{x} \mid Y = c)$ or posterior probability function $\eta_n(\boldsymbol{x}) = P_n(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$.
- Kernel bandwidth h becomes important.

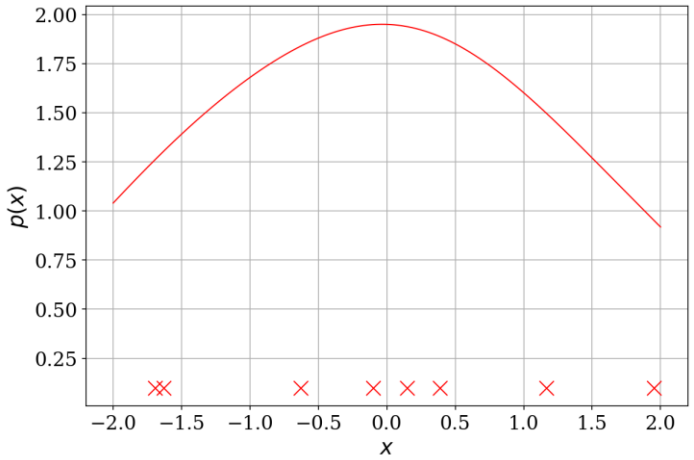


Kernal Density Estimation

Important: Selecting the *right* amount of smoothing.







Non-Parametric Plugin Classification

- Given training data $S_n = \{(\boldsymbol{X}_1, Y_1), \ldots, (\boldsymbol{X}_n, Y_n)\}$, consider estimates $\eta_n(\boldsymbol{x})$ of the posterior-probability function:

$$\eta_n(x) = \sum_{i=1}^{n} W_{n,i}(\boldsymbol{x}, \boldsymbol{X}_1, \boldsymbol{X}_2, \ldots \boldsymbol{X}_n)\, I_{Y_i=1},$$

  where

$$W_{n,i}(\boldsymbol{x}, \boldsymbol{X}_1, \boldsymbol{X}_2, \ldots \boldsymbol{X}_n) \geq 0 \text{ and } \sum_{i=1}^{n} W_{n,i}(\boldsymbol{x}, \boldsymbol{X}_1, \boldsymbol{X}_2, \ldots \boldsymbol{X}_n) = 1, \text{ for all } \boldsymbol{x} \in R^d.$$

- Weight can change from point to point as it is a function of $\boldsymbol{x}$.
- Weights also depend on the entire spatial configuration of the training feature vectors.
- The plug-in classifier is given by (1):

$$\psi_n(\boldsymbol{x}) = \begin{cases} 1, & \eta_{n,i}(\boldsymbol{x}) > 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

$$= \begin{cases} 1, & \sum_{i=1}^{n} W_{n,i}(\boldsymbol{x})\, I_{Y_i=1} > \sum_{i=1}^{n} W_{n,i}(\boldsymbol{x})\, I_{Y_i=0}, \\ 0, & \text{otherwise.} \end{cases}$$

Non-Parametric Plugin Classification

- Adding the *influences* of each data point $(\boldsymbol{X}_i, Y_i)$ on $\boldsymbol{x}$ and assigning the label of the most *influent* class.
- Typical $W_{n,i}(\boldsymbol{x})$ is inversely related to the Euclidean distance $\|\boldsymbol{x} - \boldsymbol{X}_i\| \Rightarrow$ the *influence* of $(\boldsymbol{X}_i, Y_i)$ decreases with increasing distance.
- Points that are spatially close to each other are more likely to come from the same class than points that are far away from each other.
- $W_{n,i}(\boldsymbol{x})$ depends on entire training data, implying the entire spatial configuration of training points can modify the *influence* of each training point on the test point.

Non-Parametric vs. Parametric Classification

Parametric Classification	Non-Parametric Classification
Assumes data follows a specific distribution	No assumptions about data distribution
Less flexible, assumes a fixed functional form	Highly flexible, adapts to complex patterns
Fixed number of parameters, independent of sample size	Parameters grow with sample size or data complexity
E.g. Logistic Regression, LDA, QDA, etc.	k-NN, Decision Trees, SVM etc.
More interpretable due to defined parameters	Less interpretable; often act as "black-box" models
Works well with smaller samples if assumptions are satisfied.	Requires larger samples to avoid overfitting and capture complex relationships effectively.
Simpler to compute	More computationally intensive
Sensitive to outliers	Robust against outliers

Histogram Classification

- Histogram rules are based on **partitions** of the feature space:

$$A(\boldsymbol{x}_1) \cap A(\boldsymbol{x}_2) = \emptyset, \text{ for all } \boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathbb{R}^d$$

$$\mathbb{R}^d = \bigcup_{\boldsymbol{x} \in R^d} A(\boldsymbol{x})$$

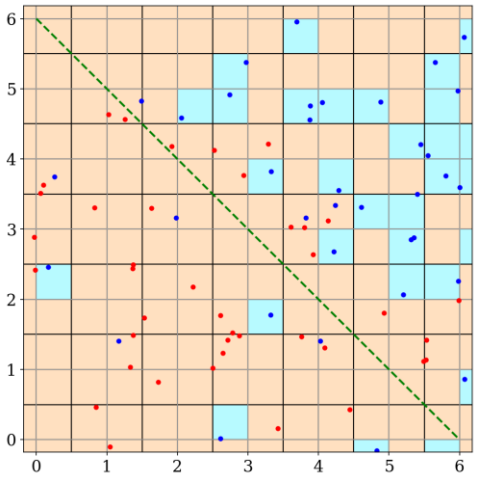
- The general histogram classification rule is the same as (1) and has weights:

$$W_{n,i}(\boldsymbol{x}) = \begin{cases} \dfrac{1}{N(\boldsymbol{x})}, & \boldsymbol{X}_i \in A(\boldsymbol{x}), \\ 0, & \text{otherwise.} \end{cases}$$

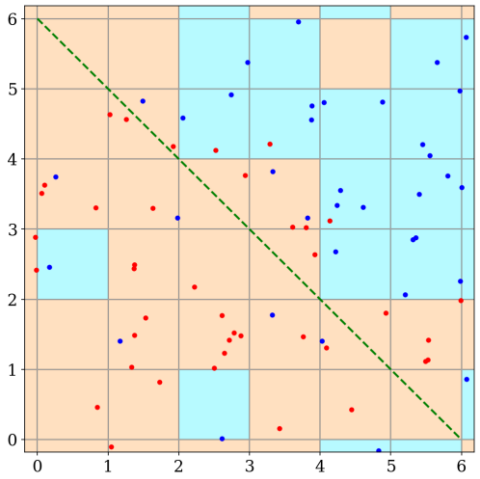
- $N(\boldsymbol{x})$ is the total number of training points in $A(\boldsymbol{x})$.
- Similar to first quantizing (discretizing) the feature space using partition and then applying the discrete histogram rule.

Cubic Histogram Rule

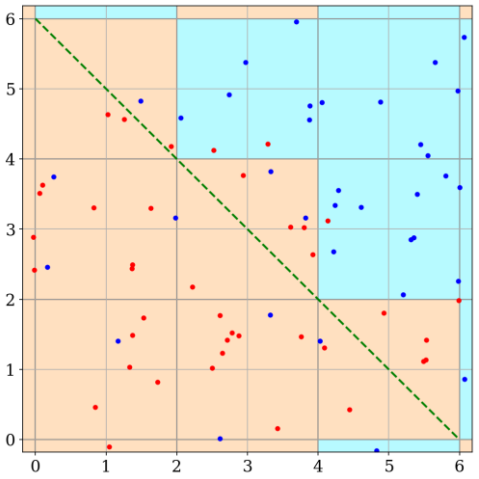
- The partition is a regular grid, and each zone is a (hyper)cube with a side of length l.
- l is the smoothing parameter.
- Too small l results in not enough points inside each zone. Too big l, estimate is too coarse.



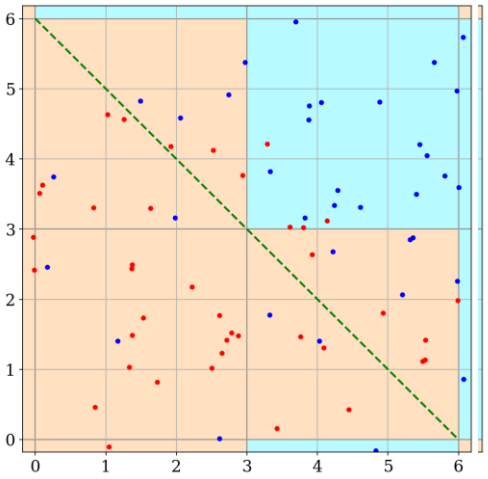
$l = 0.5$



$l = 1$



$l = 2$



$l = 3$

k-Nearest Neighbor Classification

- Oldest and widely known non-parametric classification rule.
- Classifying by labelling the majority label of k closest training points.
- Using odd values of k reduces the possibility of ties in making the classification.
- If $k = 1$, we get the nearest neighbor classification rule, which:
  - is surprisingly good for large sample size, and
  - has poor performance on small sample sizes (overfitting).
- kNN performs poorly also in cases of substantial class overlap.
- Theoretical, and empirical evidences suggest $k = 3$ and $k = 5$ perform much better than $k = 1$ for both small and large sample sizes.

k-Nearest Neighbor Classification

- The weights for the kNN classification rule of the form given in (1) are given by:

$$W_{n,i}(\boldsymbol{x}) = \begin{cases} \frac{1}{k}, & \boldsymbol{X}_i \text{ is among the } k \text{ nearest neighbors of } \boldsymbol{x}, \\ 0, & \text{otherwise.} \end{cases}$$

- When weights that assign different importance according to the distance rank is used, we get **weighted kNN** classification rule.
- Distances other than Euclidean distance could be used.
- The choice of k is an example of **hyperparameter tuning** or **model selection**
- Typically performed by using **cross-validation**.
- In regression, prediction is the (weighted) average of the values of the k samples.

k-NN Distance Measures

- **Euclidean Distance**

$$distance = \sqrt{\sum_{i=1}^{d} (x_i - \hat{x}_i)^2}$$

- **Manhattan Distance**

$$distance = \sum_{i=1}^{d} |x_i - \hat{x}_i|$$

- **Minkowski Distance**

$$distance = \left(\sum_{i=1}^{d} (|x_i - \hat{x}_i|)^p \right)^{\frac{1}{p}}$$

- Note that Minkowski distance is a generalized form of Euclidean distance ($p = 2$) and Manhattan distance ($p = 1$)

k-NN Distance Measures

- **Mahalanobis distance**

$$distance = \sqrt{(\boldsymbol{X} - \boldsymbol{\mu})^T \cdot \boldsymbol{\Sigma}^{-1} \cdot (\boldsymbol{X} - \boldsymbol{\mu})}$$

where

$\boldsymbol{X} \rightarrow$ testing point

$\boldsymbol{\mu} \rightarrow$ vector mean of independent variables

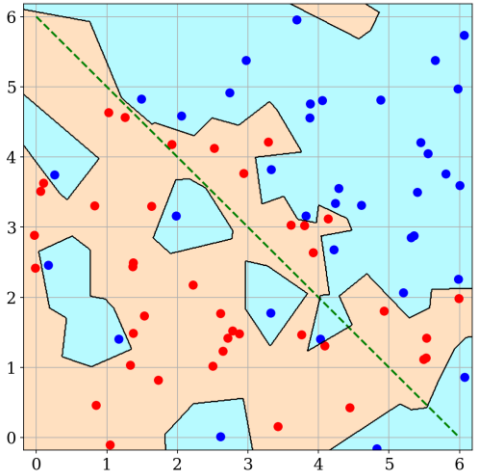
$\boldsymbol{\Sigma}^{-1} \rightarrow$ Covariance inverse matrix of independent variables

- **Hamming Distance** (For discrete/categorical features)
  - Counts the number of positions at which two strings differ. For example, hamming distance is 4 for the binary case below:

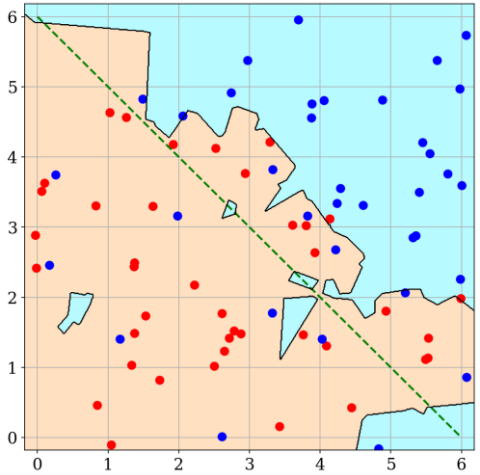
$$\boldsymbol{X} = (1\ 0\ 0\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 1)$$
$$\hat{\boldsymbol{X}} = (1\ 1\ 0\ 1\ 1\ 0\ 0\ 0\ 1\ 0\ 1)$$

k-Nearest Neighbor Classification

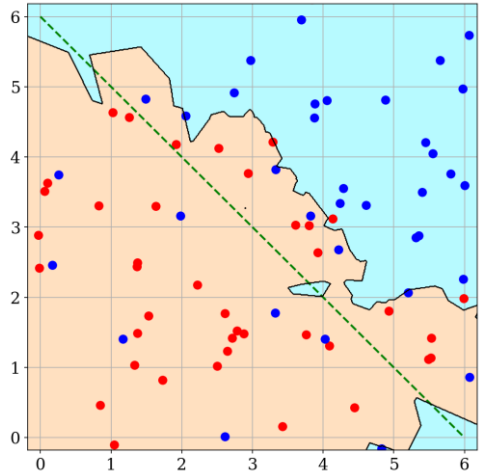
- The decision boundary is complex, especially for $k = 1$.
- 1-NN overfits the data because of small sample size and overlapping classes.
- $k = 3$ and $k = 5$ produce better results.
- $k = 7$ improves the classification but not much better than above two.



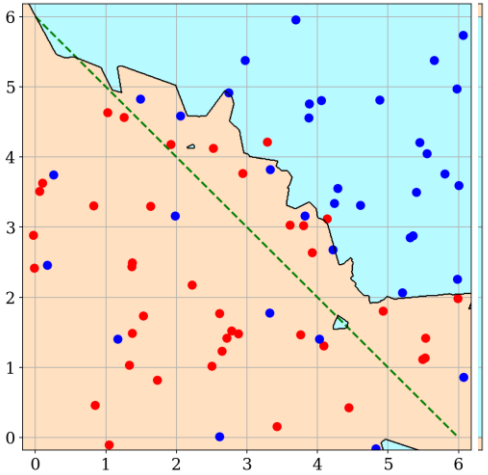
$k = 1$



$k = 3$



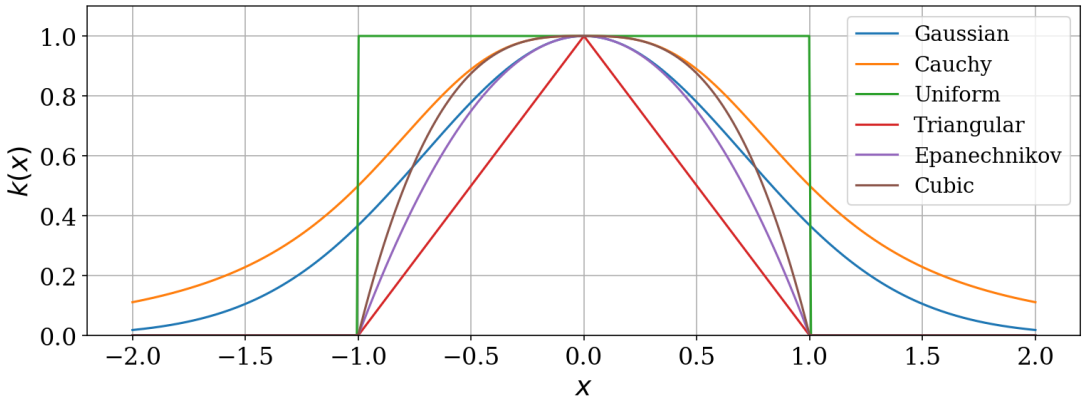
$k = 5$



$k = 7$

Kernel Classification

- Kernel rules: very general non-parametric classification rules, where weighting functions are expressed in terms of kernels.
- A **kernel** is a non-negative function $k: \mathbb{R}^d \to \mathbb{R}$ such as:
  - **Gaussian kernel**: $k(\boldsymbol{x}) = e^{-\|\boldsymbol{x}\|^2}$
  - **Cauchy kernel**: $k(\boldsymbol{x}) = \frac{1}{1+\|\boldsymbol{x}\|^{d+1}}$
  - **Uniform kernel**: $k(\boldsymbol{x}) = \boldsymbol{I}_{\{\|\boldsymbol{x}\| \leq 1\}}$
  - **Triangular kernel**: $k(\boldsymbol{x}) = (1 - \|\boldsymbol{x}\|)\boldsymbol{I}_{\{\|\boldsymbol{x}\| \leq 1\}}$
  - **Epanechnikov kernel**: $k(\boldsymbol{x}) = (1 - \|\boldsymbol{x}\|^2)\boldsymbol{I}_{\{\|\boldsymbol{x}\| \leq 1\}}$
  - **Cubic kernel**: $k(\boldsymbol{x}) = (1 - \|\boldsymbol{x}\|^3)\boldsymbol{I}_{\{\|\boldsymbol{x}\| \leq 1\}}$
- Except Gaussian and Cauchy kernels, all of them have bounded support.
- A kernel that is a monotonically decreasing function of $\|\boldsymbol{x}\|$ is called a **radial basis function (RBF).**



Kernel Classification

- The kernel classification rule is in the form (1) with weighting function:

$$W_{n,i}(\boldsymbol{x}) = \frac{k\left(\frac{\boldsymbol{x}-\boldsymbol{X}_i}{h}\right)}{\sum_{j=1}^{n} k\left(\frac{\boldsymbol{x}-\boldsymbol{X}_j}{h}\right)}$$

  where the smoothing parameter h is the kernel bandwidth.

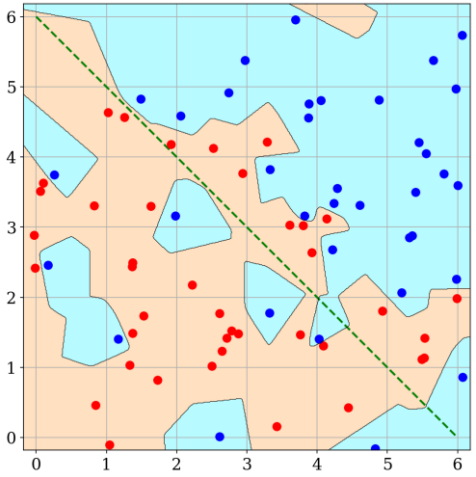
- The non-parametric plugin classifier is given by:

$$\psi_n(\boldsymbol{x}) = \begin{cases} 1, & \sum_{i=1}^{n} k\left(\frac{\boldsymbol{x}-\boldsymbol{X}_i}{h}\right) I_{Y_i=1} \geq \sum_{i=1}^{n} k\left(\frac{\boldsymbol{x}-\boldsymbol{X}_i}{h}\right) I_{Y_i=0}, \\ 0, & \text{otherwise.} \end{cases}$$

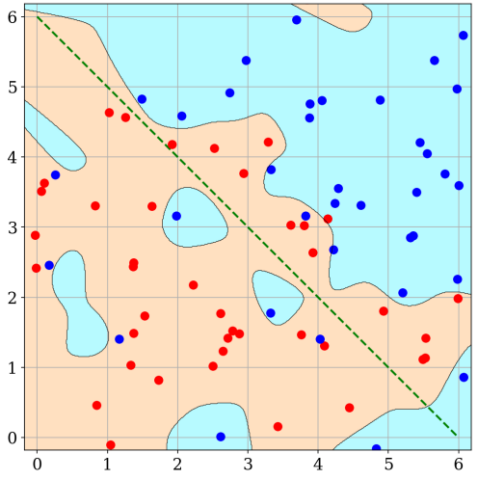
- The class with larger cumulative kernel values at the test point $\boldsymbol{x}$ assigns its label to $\boldsymbol{x}$.

Gaussian-RBF Kernel Classification

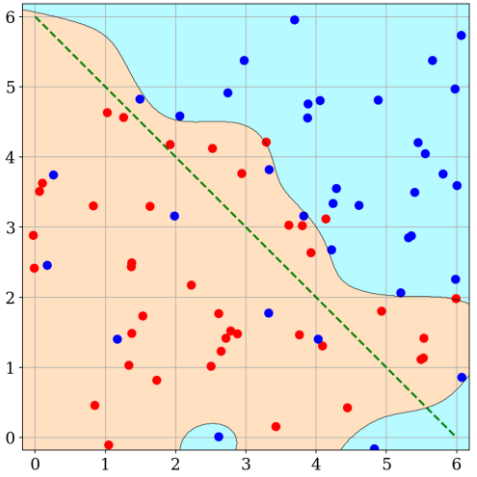
- Decision boundaries smoothen as bandwidth h increases.
- For $h = 0.1$ and $h = 0.3$, the rule is too local leading to overfitting.
- For $h = 1$, the decision boundary is very close to the optimal one.
- For larger values of h, kernel rule becomes global leading to underfitting.



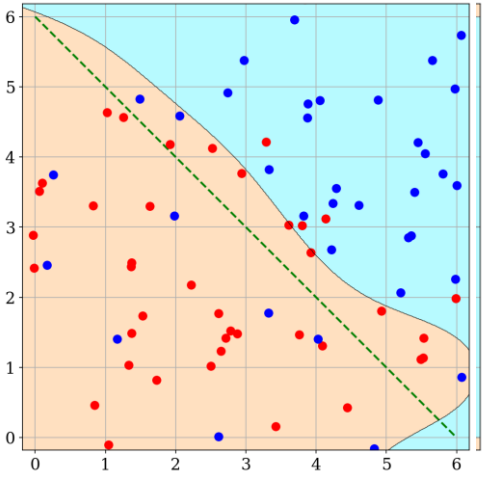
$h = 0.1$



$h = 0.3$



$h = 0.5$



$h = 1$

Kernel Classification (Contd.)

$$\eta_n(x) = \sum_{i=1}^{n} W_{n,i}(\boldsymbol{x})\, I_{Y_i=1}$$

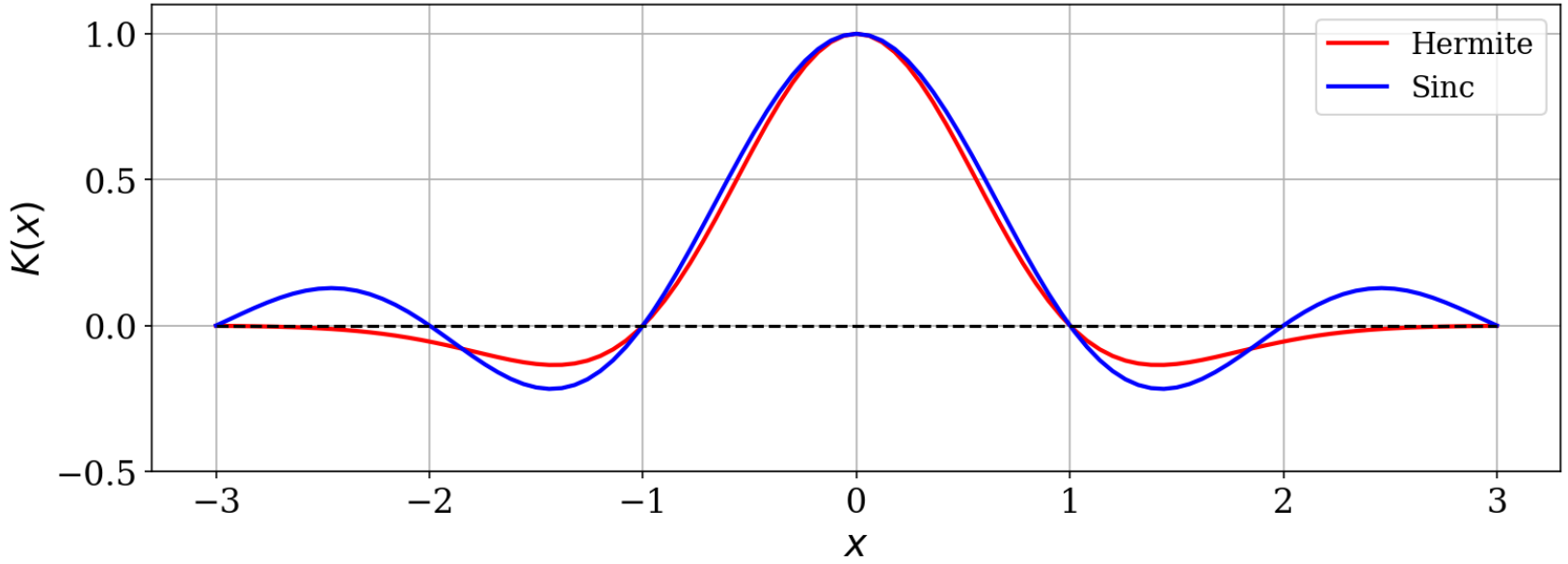
- Above requires $k(\boldsymbol{x}) \geq 0, \forall\, \boldsymbol{x} \in \mathbb{R}^d$ to make sure $\eta_n(x)$ is non-negative.'
- However, $k(\boldsymbol{x})$ below doesn't have to be non-negative.

$$\psi_n(\boldsymbol{x}) = \begin{cases} 1, & \sum_{i=1}^{n} k\left(\frac{\boldsymbol{x}-\boldsymbol{X}_i}{h}\right) I_{Y_i=1} \geq \sum_{i=1}^{n} k\left(\frac{\boldsymbol{x}-\boldsymbol{X}_i}{h}\right) I_{Y_i=0}, \\ 0, & \text{otherwise.} \end{cases}$$

- Example:
  - **Hermite kernel**: $k(\boldsymbol{x}) = (1 - \|\boldsymbol{x}\|^2)e^{-\|\boldsymbol{x}\|^2}$
  - **Sinc kernel**: $k(\boldsymbol{x}) = \frac{\sin(\pi\|\boldsymbol{x}\|)}{\pi\|\boldsymbol{x}\|}$

Kernel Classification

- There are problems where Hermite kernel outperforms all non-negative kernels.
- Classification is different from density estimation.
- A good density estimation requires more conditions and data than a good classifier design.



Thank You